

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS:

Total Marks: 30

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

- (a) Analyze the possible reasons for lack of responsiveness in the existing layout.
- (b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
- (c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
- (d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
- (e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.

- (a) Analyze the issues in the existing form validation mechanism.
- (b) Design input fields for email and phone number with appropriate HTML attributes.
- (c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
- (d) Develop a mechanism to display **real-time error messages** for invalid inputs.
- (e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

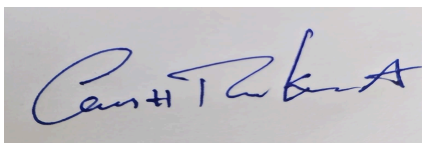
Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.

- (a) Analyze the CSS properties responsible for layout overlapping issues.
- (b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
- (c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
- (d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
- (e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

*I **Abhuri Ganesh Ram Kamal 192311183** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

A handwritten signature in blue ink, appearing to read 'Ganesh Ram Kamal', is written on a light gray rectangular background.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding ; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1A.

Aim: To design and implement a responsive webpage using CSS Flexbox and Media Queries that adapts to mobile, tablet, and desktop screen sizes.

(a) Analysis of Lack of Responsiveness

The existing website may not be responsive due to the following reasons:

1. Fixed width values (e.g., width:1200px) instead of percentage or flexible units.
2. Absence of media queries for different screen sizes.
3. Improper use of margins and padding causing content overflow.
4. Images without responsive properties (max-width:100%).
5. Using old layout methods like tables instead of Flexbox or CSS Grid.
6. No viewport meta tag in the HTML file.

(b) Responsive Webpage Layout

HTML (index.html)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Responsive Layout</title>
```

```
  <link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
<header>
```

```
  <h1>Retail Website</h1>
```

```
</header>
```

```
<nav>
```

```
<a href="#">Home</a>

<a href="#">Products</a>

<a href="#">Offers</a>

<a href="#">Contact</a>

</nav>

<div class="container">

  <aside class="sidebar">

    <h3>Categories</h3>

    <p>Electronics</p>

    <p>Fashion</p>

    <p>Furniture</p>

  </aside>

  <main class="content">

    <h2>Welcome</h2> <p>

      This is a responsive webpage designed using CSS Flexbox.

      The layout automatically adjusts for mobile, tablet,

      and desktop screens. </p>

    </main></div><footer>

    <p>© 2026 Retail Website</p></footer>

  </body>

</html>
```

CSS (style.css)

```
* {
```

```
    margin:0;
    padding:0;
    box-sizing:border-box;
    font-family:Arial,sans-serif;
}
body{
    background:#f4f4f4;
}

header{
    background:#2c3e50;
    color:white;
    text-align:center;
    padding:20px;
}
nav{
    display:flex;
    justify-content:center;
    background:#34495e;
}
nav a{
    color:white;
    text-decoration:none;
    padding:15px 20px;
}
nav a:hover{
    background:#1abc9c;
}
.container{
    display:flex;
    gap:20px;
    padding:20px;
}
.sidebar{
    flex:1;
    background:white;
    padding:20px;
}
.content{
```

```

    flex:3;
    background:white;
    padding:20px;
}
footer{
    background:#2c3e50;
    color:white;
    text-align:center;
    padding:15px;
}
/* Tablet */
@media(max-width:768px){

    .container{
        flex-direction:column;
    }
    nav{
        flex-wrap:wrap;
    }
}
/* Mobile */
@media(max-width:480px){
    nav{
        flex-direction:column;
        text-align:center;
    }
    nav a{
        border-bottom:1px solid white;
    }
    header{
        font-size:18px;
    }
    .content,
    .sidebar{
        padding:15px;
    }
}}

```

(c) Layout Adaptation Using Media Queries

Desktop (Width > 768px)

- Navigation displayed horizontally.
- Sidebar and content displayed side by side.
- Larger spacing between sections.

Tablet (Width ≤ 768px)

- Sidebar moves below the content.
- Navigation wraps automatically.
- Layout becomes single-column.

Mobile (Width ≤ 480px)

- Navigation changes to vertical.
- Reduced padding and font size.
- Content occupies full width for easy reading.

(d) Spacing, Alignment, and Distribution

The following CSS properties improve the layout:

- display: flex; for flexible layout.
- justify-content: center; to center navigation items.
- gap: 20px; to provide equal spacing.
- padding: 20px; for internal spacing.
- box-sizing: border-box; for proper element sizing.
- flex: 1 and flex: 3 for proportional distribution of space.

These techniques ensure consistent spacing and alignment across all devices.

(e) Evaluation

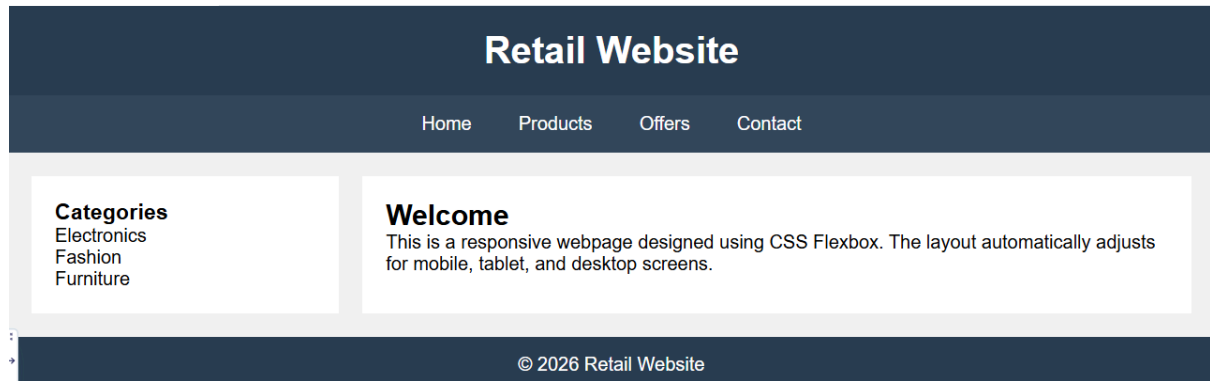
Effectiveness

- Responsive on mobile, tablet, and desktop.
- Flexible layout without horizontal scrolling.
- Proper spacing and alignment improve readability.
- Easier navigation on small screens.
- Better user experience across different devices.

Further Improvements

- Use CSS Grid for more complex layouts.
- Add responsive images using max-width:100%.
- Include animations and hover effects.
- Improve accessibility using semantic HTML and ARIA labels.
- Optimize loading speed with compressed images and minified CSS.

Result



A responsive webpage was successfully designed using **CSS Flexbox** and **Media Queries**. The layout adapts effectively to different screen sizes while maintaining proper spacing, alignment, and visual consistency.

2A.

Aim:To design and implement a registration form with JavaScript validation using Regular Expressions (Regex) for email and phone number inputs.

(a) Analysis of Existing Form Validation Issues

The existing form may have the following problems:

1. Invalid email addresses are accepted without checking the correct format.
2. Phone numbers containing letters or incorrect lengths are accepted.
3. No real-time validation, so users only notice errors after submission.
4. Error messages are unclear or not displayed.
5. Lack of client-side validation leads to incorrect data being submitted.

(b) HTML Form with Email and Phone Fields

index.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>

  <title>Registration Form</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

  <div class="container">

    <h2>Registration Form</h2>

    <form>

      <label>Email</label>

      <input type="email" id="email" placeholder="Enter Email"
onkeyup="validateEmail()">

      <span id="emailError"></span>

      <br><br>

      <label>Phone Number</label>

      <input type="text" id="phone" placeholder="Enter Phone Number"
onkeyup="validatePhone()">

      <span id="phoneError"></span>

      <br><br>

      <input type="submit" value="Register">

    </form>

  </div>

  <script src="script.js"></script>

</body>

</html>
```

CSS (style.css)

```

body{

    font-family:Arial;

    background:#f2f2f2;

}.container{

    width:350px;

    margin:50px auto;

    background:white;

    padding:20px;

    border-radius:8px;}

input{

    width:100%;

    padding:10px;

    margin-top:5px;}

span{

    color:red;

    font-size:14px;}

```

(c) JavaScript Validation using Regular Expressions

script.js

```

function validateEmail(){

    let email=document.getElementById("email").value;

    let pattern=/^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$/;

    if(pattern.test(email)){

        document.getElementById("emailError").innerHTML="Valid Email";

        document.getElementById("emailError").style.color="green"; }

```

```

else{

    document.getElementById("emailError").innerHTML="Invalid Email";

    document.getElementById("emailError").style.color="red";  }}

function validatePhone(){

    let phone=document.getElementById("phone").value;

    let pattern=/^[6-9][0-9]{9}$/;

    if(pattern.test(phone)){

        document.getElementById("phoneError").innerHTML="Valid Phone
Number";

        document.getElementById("phoneError").style.color="green";  }

    else{

        document.getElementById("phoneError").innerHTML="Invalid Phone
Number";

        document.getElementById("phoneError").style.color="red";

    }}

```

(d) Real-Time Error Message Mechanism

- The onkeyup event triggers validation whenever the user types.
- If the input matches the regular expression, a green **"Valid"** message is displayed.
- If the input is invalid, a red error message appears immediately.
- This helps users correct mistakes before submitting the form.

(e) Evaluation

Effectiveness

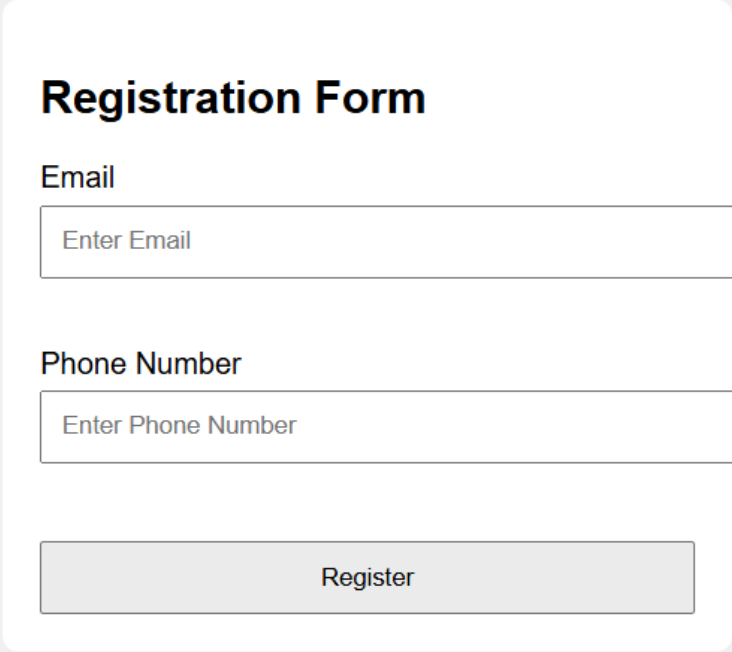
- Prevents invalid email formats.
- Ensures phone numbers contain exactly 10 digits.
- Provides instant feedback through real-time validation.
- Improves data accuracy before form submission.

- Enhances overall user experience.

Suggested Enhancements

- Disable the **Register** button until all fields are valid.
- Add server-side validation for improved security.
- Validate additional fields such as password and username.
- Display user-friendly error messages with icons.
- Sanitize inputs to protect against malicious data and attacks.

Result



The image shows a registration form titled "Registration Form". It contains two input fields: "Email" with a placeholder "Enter Email" and "Phone Number" with a placeholder "Enter Phone Number". Below these fields is a "Register" button. The form is styled with a light gray background and rounded corners.

A registration form was successfully developed using **HTML**, **CSS**, and **JavaScript**. Regular Expressions were used to validate email and phone number inputs, while real-time error messages improved data accuracy, usability, and user experience.

3A.

Aim: To identify and fix CSS layout overlapping issues by using the CSS Box Model, Flexbox, and responsive positioning techniques.

(a) Analysis of Layout Overlapping Issues

The webpage layout may overlap due to the following reasons:

1. Using fixed widths and heights instead of flexible units.
2. Incorrect use of position: absolute, causing elements to overlap.
3. Missing responsive layout techniques such as Flexbox or Grid.
4. Improper margin and padding values.
5. No media queries to adjust the layout for different screen sizes.

(b) Role of the CSS Box Model

The CSS Box Model consists of four parts:

- **Content:** The actual text or image inside the element.
- **Padding:** Space between the content and the border.
- **Border:** The outline surrounding the padding and content.
- **Margin:** Space outside the border that separates elements from each other.

Proper use of the box model ensures correct spacing, alignment, and prevents overlapping of webpage elements.

(c) Debugging the Layout

HTML (index.html)

```
<!DOCTYPE html>

<html>

<head>

  <title>CSS Layout Debugging</title>

  <link rel="stylesheet" href="style.css">

</head>

<body>

<header>

  <h2>Website Header</h2>

</header>

<div class="container">
```

```
<aside class="sidebar">
```

 Sidebar

```
</aside>
```

```
<section class="content">
```

 Main Content Area

```
</section>
```

```
</div>
```

```
<footer>
```

 Footer

```
</footer>
```

```
</body>
```

```
</html>
```

CSS (style.css)

```
*{
```

```
    margin:0;
```

```
    padding:0;
```

```
    box-sizing:border-box;
```

```
    font-family:Arial;
```

```
}
```

```
body{
```

```
    background:#f2f2f2;
```

```
}
```

```
header,footer{
```

```
    background:#2c3e50;
```

```
    color:white;

    text-align:center;

    padding:20px;
}

.container{

    display:flex;

    gap:20px;

    padding:20px;
}

.sidebar{

    flex:1;

    background:#d6eaf8;

    padding:20px;
}

.content{

    flex:3;

    background:white;

    padding:20px;
}

/* Tablet */

@media(max-width:768px){

.container{

    flex-direction:column;
}
```



```
}

/* Mobile */

@media(max-width:480px){

header,footer{

    font-size:18px;

}

.container{

    padding:10px;

}

}
```

(d) CSS Modifications to Prevent Overlapping

The following changes were made to fix the layout:

1. Used display: flex for automatic alignment.
2. Applied box-sizing: border-box to include padding and border within element width.
3. Replaced fixed widths with flex properties.
4. Added gap to create spacing between elements.
5. Used media queries to stack elements vertically on smaller screens.
6. Avoided unnecessary position: absolute and used the default static positioning.

These modifications ensure that the layout remains properly aligned and does not overlap when the browser window is resized.

(e) Evaluation

Effectiveness

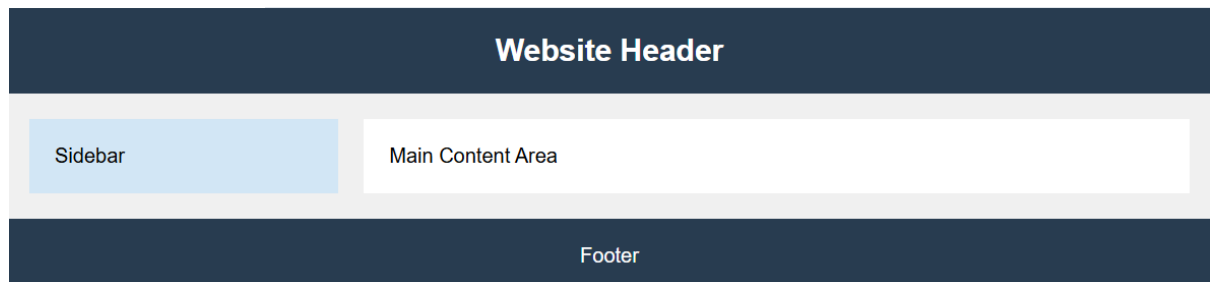
- Layout adapts correctly to different screen sizes.
- No overlapping of content during resizing.
- Proper spacing and alignment improve readability.
- Responsive design provides a better user experience.

- Code is easier to maintain using Flexbox.

Best Practices

- Use Flexbox or CSS Grid for modern layouts.
- Apply box-sizing: border-box to all elements.
- Avoid excessive use of absolute positioning.
- Use percentage or flexible (flex) units instead of fixed pixel widths.
- Test the webpage on mobile, tablet, and desktop using media queries.

Result



The webpage layout was successfully debugged by applying the CSS Box Model, Flexbox, and responsive media queries. The corrected layout prevents overlapping, maintains proper alignment, and provides a consistent user experience across different screen sizes.

